

Data Structures

Array

An array is a sequence of elements $a_1, a_2, \dots, a_n$. These elements are stored in consecutive memory cells, all have the same data type and the number of elements is fixed when we declare the array.

a	b	d	f	g	h	k	l
0	1	2	3	4	5	6	7

index

Data access = fast
Insertion/deletion = slow

to maintain contiguity, all subsequent elements have to be moved to the prev/next memory location when deleting from/inserting to the middle of an array

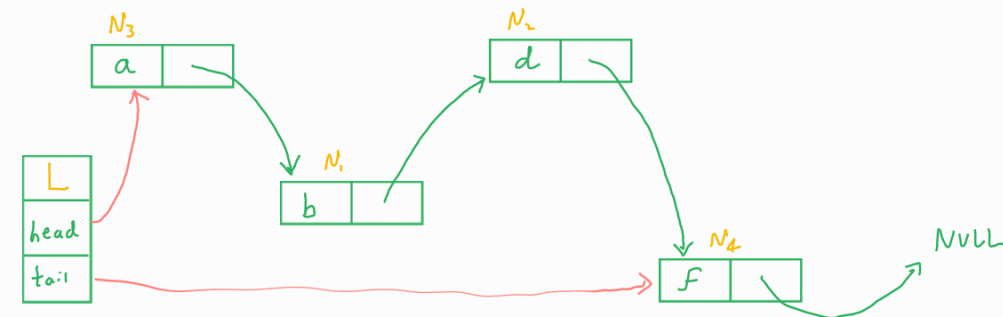
Linked Lists

A linked list is made up of nodes, each storing a piece of data and a link to the next node.

For a linked list L , L -head points to the first node, L -tail points to the last node and L -size is the number of nodes.

For a node N , N -data is the piece of data stored in the node and N -next is a pointer to the next node.

L -tail.next = NULL to signify the end of the list.



Operations:

Find:

Input: list L , positive integer i
Output: i th piece of data in L
if $i > L$.size then
 return out of range
end if
 $N = L$.head
if $i = 1$ then
 return N .data
end if
for $j = 2$ to i do
 $N = N$.next
end for
return N .data

Insertion:

Insertion of v after N
Input: list L , data v , node N
Output: L with v inserted after N
node $M \rightarrow$ new node containing the data v to be inserted
 M .data = v
 M .next = N .next
 N .next = M
if L .tail = N then
 L .tail = M
end if

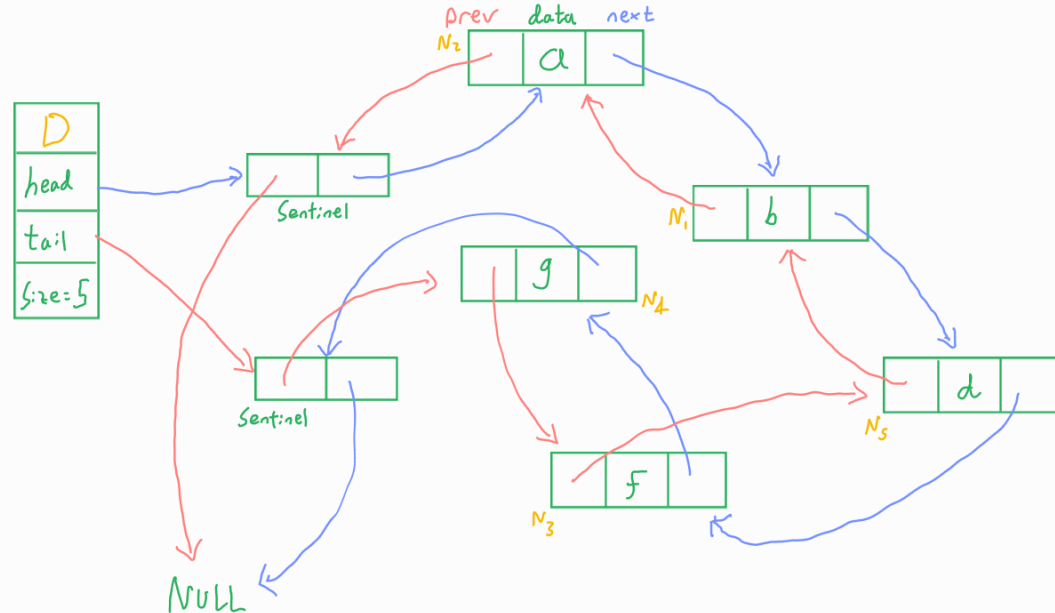
Deletion of head:

Deletion of the head
Input: list L
Output: L with head deleted
if L .head = NULL then
 return no head to delete
end if
if L .head = L .tail then
 L .head = NULL
 L .tail = NULL
 L .size = L .size - 1
end if
 L .head = L .head.next
 L .size = L .size - 1

Note: Deleting from an arbitrary position i requires traversing the list to find element $i-1$ and update the node's next pointer.

Doubly Linked Lists

A doubly linked list is one where each node has a previous link as well as a next link. Head and tail sentinel nodes (nodes without data) are also added - the head's prev and the tail's next both point to NULL. An empty doubly linked list only contains these sentinel nodes pointing to each other. The head and tail pointers for a DLL, D , point to the sentinels, and D .size excludes them from the node count.



This solves the problem of having to traverse the entire list to delete from the tail:

Deletion from doubly linked list
Input: list L, node N
Output: L with N removed
M = N.prev
P = N.next
M.next = P
P.prev = M
L.size = L.size - 1

Data access = Slow
Insertion/deletion = fast.

Stacks

A **stack** is a collection of objects that are inserted and removed according to **Last in, First out**, i.e. only the most recently inserted item can be removed, i.e. items can only be **pushed to** or **popped from** the **top** of the stack.



Operations:
Note: Stack implemented as **N-element array S** with an **integer t** giving the index of the **top** element of the stack, set to -1 if stack empty.

Basis

size
return t+1
isEmpty
return (t<0)
top
if isEmpty then throw a EmptyStackException
end if return S[t]

Push- add an element to the stack

push(e)
if size = N then throw a FullStackException
end if t = t + 1 S[t] = e

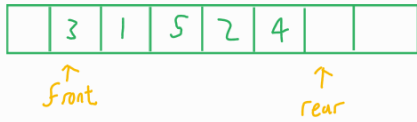
Pop- remove and return the top element

pop
if isEmpty then throw a EmptyStackException
end if e = S[t] S[t] = NULL t = t - 1 return e

This array based implementation is time efficient — all methods are $O(1)$, however since arrays have a fixed size N , if we have a large N we could be wasting memory but if we have a small N we would get stack overflow errors. Stacks can also be implemented using linked lists

QUEUES

A queue is a collection of objects inserted and removed according to **first in first out**, i.e. elements are accessed and deleted from the **front** and elements are inserted at the **rear** of the queue.



Operations:

Queues can be implemented using arrays, with two pointers: **front** pointing to the element at the front of the queue, and **rear** pointing to the next available position where an element will be enqueued. If the queue is empty, **front = rear**. After each enqueue, increment **rear**; after each dequeue increment **front**. We let both pointers wrap around.

Basics

size
return $(r - f) \bmod N$
isEmpty
return $(f = r)$
front
if isEmpty then throw a EmptyQueueException
end if
return $Q[f]$

Enqueue

enqueue(e)
if size = $N - 1$ then throw a FullQueueException
end if
$Q[r] = e$
$r = r + 1 \bmod N$

Dequeue

dequeue
if isEmpty then throw a EmptyQueueException
end if
temp = $Q[f]$
$Q[f] = \text{NULL}$
$f = f + 1 \bmod N$
return temp

Queues can also be implemented using a linked list.

Hash Tables

A hash table consists of a bucket array A of size N and a hash function h , which maps each key to an integer in the range $0 \dots N-1$, i.e. we use h to find the index $A[h(k)]$ in the hash table to store the key-value pair (k, v) . The aim of h is to seemingly randomly distribute the keys across the table, ideally uniformly, while minimising collisions.

When two keys map to the same index, this is a collision. There are several methods for handling collisions:

- Separate chaining: each bucket $A[i]$ contains a linked list — collisions are simply added to the corresponding list.
 - Performance = not great — traversing a linked list is slow so if many collisions occur performance is shite.
 - memory space is an issue
- Linear Probing: If a collision occurs, check if the next bucket is empty until you find one that is.
 - Can suffer from clustering, which affects search as search algos have to keep checking adjacent buckets until an empty one is found, so if elements all end up clustering together this will impact search performance.
- Quadratic Probing — if $A[h(k)]$ occupied, repeatedly check $A[h(k)+1]$, $A[h(k)+2^2]$ + ... + $A[h(k)+i^2]$ until a free bucket is found.
 - Can suffer from secondary clustering and may not find some free buckets at all.

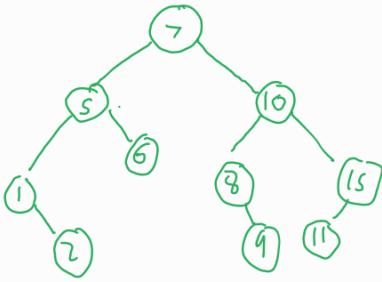
- Double hashing - If $A[h(k)]$ occupied, iteratively check $A[(h(k) + F(j)) \bmod n]$ for $j = 0, \dots, n$ where $F(j) = j \times h'(k)$, where $h'(k)$ is the secondary hash function $\rightarrow$ this should never go to zero as then we're stuck.
- Running time increases by having to evaluate a second hash function

Tombstones: When deleting from a hash table, a **tombstone** should be left in the bucket instead of leaving it empty. This is for if a collision occurs but the element causing the collision is later deleted, as otherwise a search will fail to find the rehashed element as it would look only where it's supposed to be, see it's empty and terminate without checking for past collisions.

The use of tombstones affects performance so the table should be periodically rehashed to clear an excess of tombstones.

Binary Search Trees

A **binary search tree** is a tree in which no node has more than two children, and the tree must be built and maintained such that for every node v , all elements in its left subtree have a smaller value and all elements in its right subtree have a greater value than v .



Operations - need to maintain BST property:

Insertion

```

class Node:
    ...
    def insert(self, data):
        """
        Insert new node with data
        @param data node data object to insert
        """
        if data < self.data:
            if self.left is None:
                self.left = Node(data)
            else:
                self.left.insert(data)
        else:
            if self.right is None:
                self.right = Node(data)
            else:
                self.right.insert(data)
  
```

Search

```

class Node:
    ...
    def lookup(self, data, parent=None):
        """
        Lookup node containing data
        @param data node data object to look up
        @param parent node's parent
        @returns node and node's parent if found or None, None
        """
        if data < self.data:
            if self.left is None:
                return None, None
            return self.left.lookup(data, self)
        elif data > self.data:
            if self.right is None:
                return None, None
            return self.right.lookup(data, self)
        else:
            return self, parent
  
```

Deletion:

- if node has no children, yeetus deletus it
- If node has one child, replace it with that child (and its subtree)
- If node has two children, copy the **largest smaller** data into it and delete that node instead
 $\rightarrow$ rightmost node in left subtree $\rightarrow$ can only have ≤ 1 children as leftmost cannot have a left child.


```

class Node:
    ...
    def delete(self, data):
        """
        Delete node containing data

        @param data node's content to delete
        """
        # get node containing data
        node, parent = self.lookup(data)
        if node is not None:
            children_count = node.children_count()

            if children_count == 0:
                # if node has no children, just remove it
                if parent.left is node:
                    parent.left = None
                else:
                    parent.right = None
                del node

            elif children_count == 1:
                # if node has 1 child
                # replace node by its child
                if node.left:
                    n = node.left
                else:
                    n = node.right
                if parent:
                    if parent.left is node:
                        parent.left = n
                    else:
                        parent.right = n
                del node

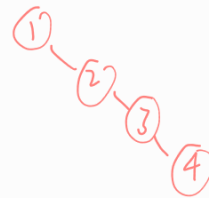
            else:
                # if node has 2 children
                # find its successor
                parent = node
                successor = node.right
                while successor.left:
                    parent = successor
                    successor = successor.left
                # replace node data by its successor data
                node.data = successor.data
                # fix successor's parent's child
                if parent.left == successor:
                    parent.left = successor.right
                else:
                    parent.right = successor.right

```

Using leftmost node in right subtree here,
could also use rightmost node in left subtree
as successor.

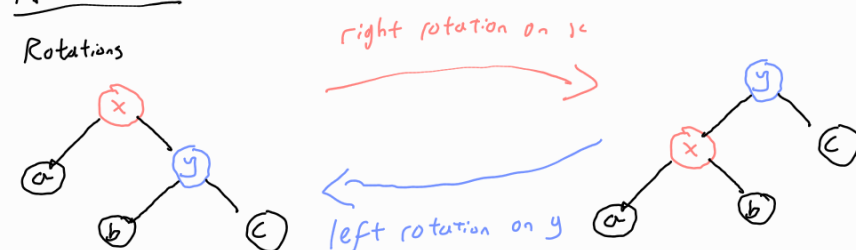
Time Complexity: $O(h) = O(\log n)$ on average

$O(n)$ in worst case (unbalanced tree) $\rightarrow$



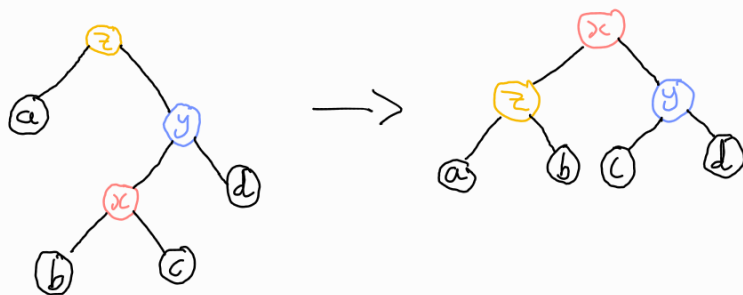
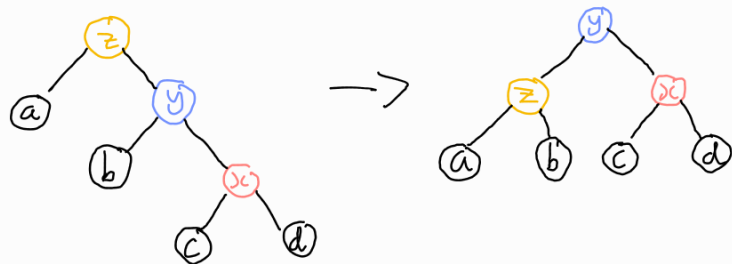
AVL Trees

Rotations



Since each rotation always performs the same number of operations, they are $O(1)$

Trinode restructures



These trinode restructures are each just a series of rotations, so are also $O(1)$

An **AVL Tree** is a **BST** with the additional property that for every node v , the heights of v 's children differ by at most 1. The rotations and trinode restructures are used to maintain this property as new nodes are added.

The height of an AVL Tree storing N elements is $O(\log n)$

Proof: let $n(h)$ be the minimum number of nodes in an AVL Tree of height h

Base case: $n(1) = 1$, $n(2) = 2$

○



For $n > 2$, the worst case AVL tree (most unbalanced) of height h contains the root node, a subtree of height $h-1$ and a subtree of height $h-2$, i.e.

$$n(h) = \underbrace{n(h-1)}_{\text{worst case subtree of height } h-1} + \underbrace{n(h-2)}_{\text{worst case subtree of height } h-2} + \underbrace{1}_{\text{root}}$$

Since $n(h-1) = n(h-2) + n(h-3) + 1$, i.e.

$$n(h-1) > n(h-2)$$

$$n(h) > 2n(h-2)$$

$$> 2 \times 2n(h-4)$$

$$> 2 \times 2 \times 2n(h-6)$$

$$n(h) > 2^i n(h-2i)$$

letting $h-2i = 2$, i.e. $i = \frac{h}{2} - 1$

$$n(h) > 2^{\frac{h}{2}-1} \times n(2) = 2^{\frac{h}{2}-1} \times 2 \quad \text{from base case}$$

$$n(h) > 2^{\frac{h}{2}}$$

$$\log(n(h)) > \frac{h}{2} \log 2$$

$$h < \frac{2 \log(n(h))}{\log 2}$$

Since $n(h)$ is the minimum no. of nodes for an AVL tree, for a general AVL tree of height h :

$$n \geq n(h)$$

Note: AVL Trees with the minimum no. of nodes for a given height are the worst case as these are the most unbalanced.

$$h < \frac{2}{\log_2} \log(n)$$

$$\therefore h \text{ is } O(\log(n))$$

Operations:

Insertion: as with BST but need to travel up the height of the tree to check the height-balance property of all nodes between the inserted node and the root — Since $h = O(\log n)$ this process is $O(\log n)$, If tri-node restructuring needed, this is $O(1)$ as it only needs to be performed at most once per insertion
 $\therefore$ insertion is $O(\log n)$ in worst case.

Deletion: Same procedures as BST involving checking the number of children of the node etc. Since deletion could reduce the height of the subtree that contained the deleted node, we may need to rebalance every level above the deleted node using tri-node restructuring in the worst case. Tri-node restructuring is $O(1)$ and there are $O(\log n)$ levels so deletion is $O(\log n)$ in worst case.

Searching: $O(\log n)$ as same procedure as BST — AVL is just a special case of BST.

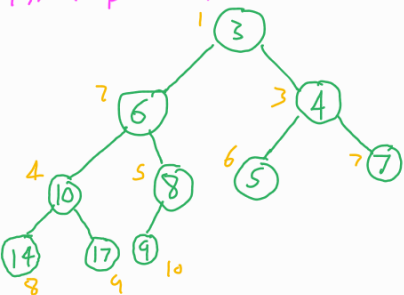
Heaps

A **Heap** is a tree structure where each node in the tree has no more than 2 children, and the tree is complete, i.e. each level is filled before a new level is started. Furthermore the **heap property** must be satisfied — the value stored at each node must be $\geq$ the values stored at its children (for **max-heaps** $\rightarrow$ if a **min-heap** then $\leq$)

Note: although heaps behave **logically** as trees, they are **physically** represented as arrays:

- the **root node** is in index 1 (for 1-indexed arrays)
- for a node at index i :
 - left** (i) is at index $2i$
 - right** (i) is at index $2i + 1$
 - parent** (i) is at index $\lfloor i/2 \rfloor$

Min-heap example



array representation

3	6	4	10	8	5	7	14	17	9	...
1	2	3	4	5	6	7	8	9	10	

Operations:

Heapify — maintaining the min/max heap property

- Start at root node v
- (for maxheap) find the largest child of v , w
- If $w > v$, swap them and recursively call Heapify on w .
- Time complexity: $O(\log n)$ (one recursive call for each level of the tree in worst case, num of levels = $O(\log n)$).

Heapify (A, v, n)

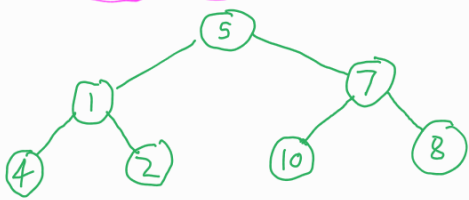
```
// n is heap size
// find largest among v and 2v (left child)
largest = v
if 2v <= n and A[2v] > A[v] then largest = 2v
// find largest among winner above and
// 2v+1 (right child)
if 2v+1 <= n and A[2v+1] > A[largest] then
  largest = 2v+1
if largest != v then
  swap A[v], A[largest]
  Heapify (A, largest, n)
```

$O(1)$

$T(\text{size of subtree}) \approx n/2$

$\therefore O(\log n)$

Example - (min) heapify

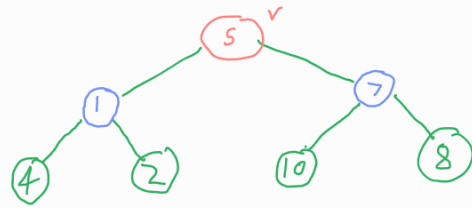


$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 5 & 1 & 7 & 4 & 2 & 10 & 8 \\ \hline \end{array}$$

1 2 3 4 5 6 7

$n = 7$

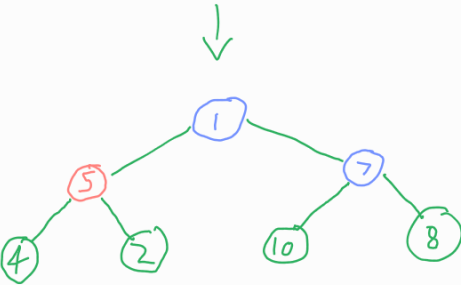
Heapify($A, v=1, n=7$)



$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline \overset{v}{5} & \overset{2v}{1} & \overset{2v-1}{7} & 4 & 2 & 10 & 8 \\ \hline \end{array}$$

1 2 3 4 5 6 7

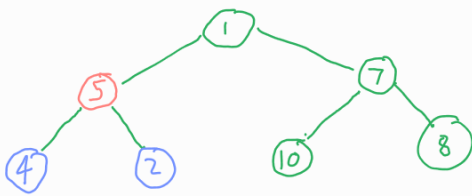
Smallest = $2v = A[2]$
Swap $A[1]$ and $A[2]$



$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 1 & 5 & 7 & 4 & 2 & 10 & 8 \\ \hline \end{array}$$

1 2 3 4 5 6 7

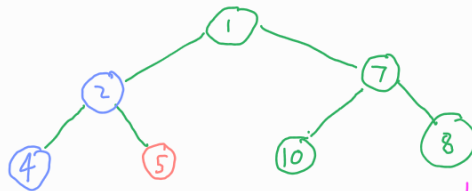
Heapify($A, v=2, n=7$)



$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline \overset{v}{1} & \overset{2v}{5} & \overset{2v+1}{7} & 4 & 2 & 10 & 8 \\ \hline \end{array}$$

1 2 3 4 5 6 7

Smallest = $2v+1 = A[5]$
Swap $A[2]$ and $A[5]$



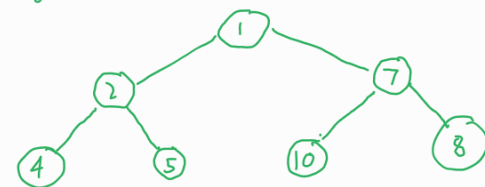
$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 1 & 2 & 7 & 4 & 5 & 10 & 8 \\ \hline \end{array}$$

1 2 3 4 5 6 7

Heapify($A, v=5, n=7$)

Exits with no recursive call as $2v$ and $2v+1$ are both $> n (=7)$

Final Minheap:



$$A = \begin{array}{|c|c|c|c|c|c|c|} \hline 1 & 2 & 7 & 4 & 5 & 10 & 8 \\ \hline \end{array}$$

1 2 3 4 5 6 7

- BuildHeap - turning an arbitrary array into a heap
- Start at **leaves** and call **Heapify** on each node to build up the heap.
 - Time Complexity is Somehow $O(n)$

BuildHeap(A, n)

```

for i=n downto 1 do
    Heapify(A, i, n)
endfor
  
```

Insertion

- Insert new element in the bottom left-most free space.
- If new element violates min/max heap property, Swap them and recurse into the parent node etc until heap property is maintained.
- Time Complexity $O(\log n)$

Deletion

- Replace the value of the element to be deleted with the value of the last node (bottom right most node) - this maintains completeness of the binary tree
- Delete last node
- Heapify the element that was replaced.
- Time Complexity $O(\log n)$